SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606 **COURSE NAME**: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting –Plotting (BL 3)

Assessment Tool Weightage: 10%

QUESTIONS: 10 **Total Marks**:20

Annexure A

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I M. Vidhya(192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

Sol)

To process continuously collected hospital patient data efficiently, I would use Pandas DataFrames for structured data manipulation and NumPy arrays for fast numerical operations.

1. File Reading and Data Loading:
Read the CSV file using pd.read_csv(). Inspect the dataset using head(), info(), and describe() to identify data types, missing values, and abnormal values.

2. Handling Missing Values:
Identify missing values using isnull().sum(). Numerical missing values can be replaced using the median or mean, while categorical missing values can be filled with the mode or "Unknown". Critical records can be removed if essential information is unavailable.

3. Removing Duplicates:
Use drop_duplicates() to remove repeated patient records. Before deleting them, important fields such as patient ID and timestamp should be checked to avoid removing valid repeated visits.

4. Standardizing Units and Formats:
Convert measurements into common units, such as converting temperature to °C and weight to kg. Date and time values can be standardized using pd.to_datetime(). Categorical values such as "Male", "male" and "M" can be standardized to one format.

5. Indexing and Filtering:
Patient ID or timestamp can be used as an index for faster access. Boolean filtering can identify valid records, for example, selecting patients whose age is greater than 0 and within a reasonable range.

6. Aggregation:
Use groupby() to calculate statistics such as average temperature, average blood pressure, or number of patients per department.

7. Handling Anomalies:
Unexpected values such as negative age, impossible blood pressure, invalid dates, or unknown categories should be flagged. Such records can be corrected, excluded, or sent for manual verification rather than silently deleting them.

8. Saving and Near Real-Time Processing:
After cleaning, save the processed data using to_csv(). For near real-time processing, files can be processed in batches or chunks using the chunksize option, reducing memory usage.

Output:
The final output is a clean, standardized and indexed patient DataFrame that can be used for reporting and analysis efficiently.

2. A financial company provides you with millions of transaction records that must be analyzed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

Sol)

For millions of financial transaction records, I would use NumPy arrays for fast numerical computation and Pandas DataFrames for filtering, grouping, indexing and statistical analysis. Since memory and computation time are limited, the processing must be optimized.

1. Efficient Data Loading:
Read only required columns using usecols in pd.read_csv(). Large files can be processed in smaller batches using chunksize, preventing the entire dataset from being loaded into memory.

2. Memory Optimization:
Use suitable numerical data types such as int32 or float32 where appropriate. Convert repeated categorical columns such as transaction type or region to the category data type to reduce memory consumption.

3. Filtering:
Use vectorized Pandas/NumPy operations instead of Python loops. For example, transactions above a particular amount can be selected using Boolean conditions.

4. Indexing:
Frequently searched columns such as customer_id, transaction_id, or date can be indexed using set_index(). Proper indexing can improve lookup and selection operations.

5. Grouping and Aggregation:
Use groupby() to calculate total spending, average transaction amount and transaction count for each customer. Functions such as sum(), mean(), count(), and std() can identify unusual spending behavior.

6. Statistical Analysis:
NumPy functions such as mean(), median(), std(), min(), and max() can be used for fast numerical calculations. Transactions significantly different from normal customer behavior can be flagged as possible anomalies.

7. Performance Optimization:
Avoid unnecessary copies of DataFrames and repeated operations. Use vectorized operations, efficient data types, chunk processing and appropriate indexes.

8. Debugging:
Performance can be checked using execution-time measurements and memory inspection. Incorrect outputs can be identified using shape, dtypes, isnull(), duplicated(), describe(), and sample records. Intermediate results should also be validated.

Output:
The result is an efficient transaction-analysis system that uses less memory and computation time while identifying unusual spending patterns accurately.

3. You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate the merged dataset and resolve conflicts such as missing or duplicate keys?

Sol)

To combine sales, marketing and customer-support datasets, I would use Pandas file I/O, merging, concatenation, grouping and sorting operations while validating the final DataFrame.

1. Reading Different File Formats:
Pandas supports different formats. CSV files can be read using pd.read_csv(), Excel files using pd.read_excel(), and JSON files using pd.read_json().

2. Standardizing Data:
Before combining the datasets, column names, data types, date formats and categorical values should be standardized. For example, Customer_ID, customer_id, and CustomerID should be converted into a common column name.

3. Handling Missing and Duplicate Keys:
Check key columns using isnull() and duplicated(). Missing customer IDs should be investigated rather than automatically deleted. Duplicate keys should be checked to determine whether they represent duplicate records or legitimate multiple transactions.

4. Combining Data:
Use pd.merge() when datasets share a common key such as customer_id. Different join types such as inner, left, right and outer joins can be selected according to the information required. pd.concat() can be used when datasets have similar structures and need to be appended.

5. Maintaining Indexing:
After merging, reset or set the index appropriately using reset_index() or set_index(). A unique customer ID can be used as an index when suitable.

6. Grouping and Sorting:
Use groupby() to calculate sales by customer, marketing response by campaign, or support requests by customer. sort_values() can arrange the final data by customer ID, date or sales amount.

7. Validation:
Compare record counts before and after merging. Check for unexpected duplicate keys, missing values, unmatched records and incorrect data types. Important totals from the original datasets should also be compared with the merged results.

8. Conflict Resolution:
If the same customer has conflicting information, apply predefined rules such as using the most recent record or the trusted department source. Conflicts should be logged for review.

Output:
The final output is a unified, validated DataFrame containing information from all departments without losing important records.

4. A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for dashboards. Describe how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.

Sol)

I would design an automated pipeline using Pandas for data processing, NumPy for numerical calculations, and Matplotlib for visualization.

1. File Handling:
The system checks a specified folder for newly received daily files. Based on the file extension, appropriate Pandas functions such as read_csv() or read_excel() are used to load the data.

2. Data Validation:
After loading, check the column names, data types, missing values and number of records. Required columns should be verified before processing.

3. Filtering:
Use Boolean conditions to select relevant records. Invalid records such as negative quantities, incorrect dates or impossible values should be filtered or flagged.

4. Data Processing:
Handle missing values using appropriate methods such as fillna() or dropna(). Convert columns to correct data types and standardize date and categorical formats.

5. Aggregation:
Use groupby() to calculate totals, averages, counts and other statistics. NumPy can perform efficient numerical calculations such as mean, standard deviation and percentile calculations.

6. Sorting:
Use sort_values() to arrange the summary data according to important fields such as sales, date or customer count. This produces dashboard-ready output.

7. Visualization:
Use Pandas plotting functions or Matplotlib to create graphs such as bar charts, line charts and histograms. These visualizations can show daily trends and important statistics.

8. Error Handling:
Use try-except blocks to handle corrupted files or incorrect formats. If required columns are missing, the file should be rejected and an error log should be generated. Unexpected data types and malformed records should also be recorded for correction.

9. Output:
The processed summary can be saved using to_csv() or to_excel(), while visualizations can be saved as image files.

Output:
The automated pipeline produces validated, processed and sorted daily reports and visualizations with minimal manual intervention.

5. You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using

Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

Answer

For real-time e-commerce clickstream data, **Pandas** can be used to clean, structure and transform the raw data, while **NumPy** can perform efficient numerical operations. The processed data can then be supplied to machine-learning models.

1. Loading and Inspection:
Read the clickstream data using pd.read_csv() or another suitable Pandas I/O function. Use head(), info(), describe() and isnull().sum() to understand the dataset.

2. Handling Missing Values:
Missing values in numerical columns can be replaced using suitable statistical values such as median. Missing categorical values can be filled with "Unknown" or the mode. Critical records may be removed if required information is missing.

3. Standardizing Formats:
Convert timestamps using pd.to_datetime(). Standardize product IDs, browser names, device types and other categorical values. Duplicate click records can be identified using duplicated() and removed when they are confirmed as duplicates.

4. Filtering:
Filter invalid or irrelevant events, such as records with invalid timestamps, unknown event types or negative values. Boolean indexing can perform these operations efficiently.

5. Feature Extraction:
New features can be created from clickstream data, such as session duration, number of pages viewed, number of clicks, average time between clicks, product-category counts and purchase indicators.

6. Indexing:
Timestamp can be used for time-based analysis, while a combination of user ID and timestamp can help organize user activity. Proper indexing makes time-based filtering and user-level analysis more efficient.

7. Aggregation:
Use groupby() to calculate user-level features such as total clicks, number of products viewed, total purchases and average session duration.

8. Preparing for Machine Learning:
Categorical variables should be encoded into numerical form, numerical features should be scaled when required, and unnecessary columns should be removed. The processed data should then be divided into training, validation and testing sets.

9. Real-Time Processing:
For continuous data, process records in small batches or chunks instead of loading the complete dataset repeatedly. This reduces memory usage and processing time.

Output:
The final DataFrame contains clean, consistent and meaningful customer-behavior features that are ready for downstream machine-learning models such as classification or recommendation systems.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**25**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand